

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA ĐIỆN – ĐIỆN TỬ



BÁO CÁO THỰC TẬP AI

BÀI 1: CLASSIFICATION – TITANIC DATASET

MÔN HỌC: THỰC HÀNH HỌC MÁY VÀ TRÍ TUỆ NHÂN TẠO

GIẢNG VIÊN HƯỚNG DẪN: PGS_TS. TRƯƠNG NGỌC SƠN

SINH VIÊN THỰC HIỆN: TRƯƠNG PHÚC THỊNH - 23161336

Tp. Hồ Chí Minh, tháng 5 năm 2026

MỤC LỤC

1. GIỚI THIỆU BÀI TOÁN	1
2. CƠ SỞ LÝ THUYẾT	1
2.1. Bài toán Classification	1
2.2. Logistic Regression	1
2.3. Decision Tree.....	2
2.4. Random Forest	2
2.5. Các chỉ số đánh giá	2
3. QUY TRÌNH THỰC HIỆN	3
3.1. Chuẩn bị môi trường và dữ liệu	3
3.2. Tiền xử lý dữ liệu	5
3.3. Xây dựng mô hình.....	6
3.4. Huấn luyện và đánh giá mô hình.....	7
3.5. Các tham số xử dụng.....	7
3.6. Công cụ và phần mềm xử dụng.....	7
4. KẾT QUẢ THỰC NGHIỆM.....	9
4.1. Kết quả Data Exploration (Bài tập 1).....	9
4.2. Kết quả Visualization (Bài tập 2)	10
4.2.1 Age distribution.....	10
4.2.2 Fare distribution	12
4.2.3 Survival theo Sex	13
4.2.4 Survival theo Pclass	14
4.3. Kết quả xử lý dữ liệu thiếu và Feature Engineering (Bài tập 3)	15
a. Thuộc tính Cabin.....	16
b. Thuộc tính Embarked.....	16

4.4. Feature Engineering (Bài tập 4)	17
a. Đặc trưng FamilySize	17
b. Đặc trưng IsAlone	17
4.5. Kết quả Title Extraction và Encoding (Bài tập 6)	18
a. Trích xuất Title	18
b. Gom nhóm các Title hiếm	19
c. Encode đặc trưng Title	19
4.6. Kết quả Correlation Analysis (Bài tập 6)	19
4.7. Kết quả so sánh các mô hình (Bài tập 7)	21
4.8. Kết quả Hyperparameter Tuning (Bài tập 8)	25
4.9. Kết quả Cross Validation (Bài tập 9)	26
4.10. Kết quả Feature Importance (Bài tập 10)	27
5. PHÂN TÍCH VÀ THẢO LUẬN	29
5.1. Nhận xét kết quả so sánh mô hình	29
5.2. Tại sao Logistic Regression hoạt động tốt?	29
5.3. Khi nào Decision Tree bị overfitting?	29
5.4. Tại sao Random Forest tốt hơn Decision Tree?	30
6. TRẢ LỜI CÂU HỎI	31
Câu 1: Vì sao Logistic Regression hoạt động tốt?	31
Câu 2: Khi nào Decision Tree bị overfitting?	31
Câu 3: Vì sao Random Forest tốt hơn Decision Tree?	31
7. KẾT LUẬN	32

DANH MỤC HÌNH ẢNH

Hình 3.1a. Kiểu dữ liệu trong tập train	4
Hình 3.2a. Kiểu dữ liệu và cấu trúc các cột trong tập dữ liệu.....	5
Hình 3.2b. Thống kê dữ liệu thiếu trong tập dữ liệu (số lượng và tỉ lệ)	6
Hình 4.1a. Kích thước tập train và test	9
Hình 4.1b. Kiểu dữ liệu các cột và thống kê missing values	10
Hình 4.2.1: Age distribution.....	11
Hình 4.2.2: Fare distribution	12
Hình 4.2.3: Survival theo Sex	13
Hình 4.2.4a: Survival theo Pclass	14
Hình 4.2.4b: Tỉ lệ sống sót.....	15
Hình 4.3. Kiểm tra kết quả sau khi xử lý dữ liệu thiếu.....	17
Hình 4.4. Kết quả feature engineering: FamilySize và IsAlone	18
Hình 4.4. Kết quả Title Extraction và Encoding	19
Hình 4.6a. Correlation Heatmap giữa các features và Survived.....	20
Hình 4.6b. Xếp hạng tương quan corr với Survived	20
Hình 4.7a. Logistic Regression – Accuracy: 79.89%	21
Hình 4.7b. Decision Tree – Accuracy: 76.54%.....	22
Hình 4.7c. Random Forest – Accuracy: 80.45%.....	22
Hình 4.7d. KNN – Accuracy: 80.45%	23
Hình 4.7e. SVM – Accuracy: 83.24%.....	23
Hình 4.7f. Biểu đồ so sánh Accuracy các mô hình	24
Bảng 4.1. Tổng hợp kết quả các mô hình (Precision / Recall / F1: class 0 / class 1)	24
Hình 4.8. GridSearchCV Best Params	26
Hình 4.9. Cross Validation Score	27

Hình 4.10a. Biểu đồ Feature Importance của Random Forest	28
Hình 4.10b. Xếp hạng chi tiết Feature Importance Score	28

1. GIỚI THIỆU BÀI TOÁN

Bài thực hành này tập trung vào bài toán phân loại (Classification) trong học máy (Machine Learning), sử dụng bộ dữ liệu Titanic nổi tiếng trên Kaggle. Mục tiêu chính là xây dựng một mô hình có khả năng dự đoán hành khách nào sống sót sau thảm họa tàu Titanic dựa trên các thông tin cá nhân như tuổi, giới tính, hạng vé và các đặc trưng khác.

Bài toán này là một ví dụ điển hình của bài toán phân loại nhị phân (binary classification): đầu ra chỉ có hai giá trị là Survived = 1 (sống sót) và Survived = 0 (không sống sót). Đây là dạng bài toán phổ biến và được ứng dụng rộng rãi trong y tế (chẩn đoán bệnh), tài chính (phát hiện gian lận), và nhiều lĩnh vực khác.

Quá trình thực hiện bài thực hành bao gồm các bước: phân tích dữ liệu ban đầu (EDA), xử lý dữ liệu thiếu, feature engineering, huấn luyện và so sánh nhiều mô hình machine learning, cũng như đánh giá hiệu năng mô hình qua các chỉ số đa dạng.

2. CƠ SỞ LÝ THUYẾT

2.1. Bài toán Classification

Classification là dạng bài toán học có giám sát (supervised learning), trong đó mô hình học từ dữ liệu được gán nhãn và dự đoán nhãn cho dữ liệu mới. Bài toán phân loại nhị phân có đầu ra là một trong hai lớp, trong khi phân loại đa lớp có từ ba lớp trở lên.

Ví dụ về bài toán Classification:

- + Phân loại email spam / không spam
- + Chẩn đoán bệnh: có bệnh / không có bệnh
- + Dự đoán sống sót trong thảm họa Titanic: 0 hoặc 1

2.2. Logistic Regression

Logistic Regression là thuật toán phân loại dựa trên hàm sigmoid. Thay vì dự đoán giá trị liên tục như Linear Regression, Logistic Regression dự đoán xác suất thuộc về một lớp nhất định. Công thức tính xác suất:

$$P(y = 1 | x) = \frac{1}{1 + e^{-w^T x}}$$

Ngưỡng phân loại thông thường là 0.5: nếu xác suất lớn hơn 0.5, mô hình dự đoán là lớp 1 (sống sót); ngược lại là lớp 0. Logistic Regression phù hợp khi mối quan hệ giữa đặc trưng và nhãn là tuyến tính

2.3. Decision Tree

Decision Tree là mô hình phân loại dạng cây quyết định. Tại mỗi nút trong cây, mô hình chia dữ liệu dựa trên giá trị của một đặc trưng để tối thiểu hóa sự không thuần nhất (impurity) của từng nhánh. Tiêu chí phổ biến là Gini impurity hoặc Information Gain.

Ưu điểm của Decision Tree là dễ hiểu và trực quan, tuy nhiên dễ bị overfitting nếu cây quá sâu. Tham số `max_depth` giúp kiểm soát độ phức tạp của cây.

2.4. Random Forest

Random Forest là kỹ thuật ensemble học kết hợp nhiều Decision Tree độc lập. Mỗi cây được huấn luyện trên một tập con ngẫu nhiên của dữ liệu (bootstrap sampling) và sử dụng một tập con ngẫu nhiên của các đặc trưng. Kết quả cuối cùng được tổng hợp bằng cách bỏ phiếu (voting) đối với bài toán phân loại.

Random Forest khắc phục hiện tượng overfitting của Decision Tree đơn lẻ nhờ vào cơ chế trung bình hóa nhiều mô hình (bagging), giúp giảm variance trong khi vẫn giữ bias ở mức thấp.

2.5. Các chỉ số đánh giá

Đối với bài toán phân loại, accuracy không phải lúc nào cũng là chỉ số đánh giá tốt nhất, đặc biệt khi dữ liệu mất cân bằng. Các chỉ số quan trọng bao gồm:

- Accuracy: tỷ lệ dự đoán đúng trên tổng số mẫu.
- Precision: trong số các mẫu được dự đoán là Positive, bao nhiêu phần trăm thực sự là Positive.

- Recall (Sensitivity): trong số các mẫu thực sự là Positive, bao nhiêu phần trăm được dự đoán đúng.
- F1-score: trung bình điều hòa của Precision và Recall, phù hợp khi cần cân bằng cả hai chỉ số trên.

3. QUY TRÌNH THỰC HIỆN

3.1. Chuẩn bị môi trường và dữ liệu

Bài thực hành được thực hiện trên nền tảng Kaggle Notebook với Python 3.12. Các thư viện sử dụng bao gồm numpy, pandas (xử lý dữ liệu), matplotlib và seaborn (trực quan hóa), sklearn (mô hình machine learning).

```
# 1. IMPORT

# =====

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score,
GridSearchCV

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier

from sklearn.neighbors import KNeighborsClassifier

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, classification_report
```

Dataset Titanic được tải trực tiếp từ Kaggle Competition với đường dẫn:

```
train = pd.read_csv('/kaggle/input/competitions/titanic/train.csv')
```



```
test = pd.read_csv('/kaggle/input/competitions/titanic/test.csv')
```

Kết quả: tập train gồm 891 mẫu $\times$ 12 đặc trưng, tập test gồm 418 mẫu $\times$ 11 đặc trưng (không có cột Survived).

train.csv (61.19 kB)

Detail Compact Column

# PassengerId	# Survived	# Pclass	Δ Name	Δ Sex	# Age	# SibSp	# Parch	Δ Ticket	# Fare
1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833
3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.925
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1
5	0	3	Allen, Mr. William Henry	male	35	0	0	373450	8.05
6	0	3	Moran, Mr. James	male		0	0	330877	8.4583
7	0	1	McCarthy, Mr. Timothy J	male	54	0	0	17463	51.8625
8	0	3	Palsson, Master. Gosta Leonard	male	2	3	1	349909	21.075
9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27	0	2	347742	11.1333

Hình 3.1a. Kiểu dữ liệu trong tập train

Các thuộc tính trong bộ dữ liệu bao gồm:

- Giới tính (*Sex*)
- Độ tuổi (*Age*)
- Hạng vé (*Pclass*)
- Giá vé (*Fare*)
- Cảng lên tàu (*Embarked*)
- Số người thân đi cùng (*SibSp*, *Parch*)
- Thông tin tên (*Name*), cabin (*Cabin*),...

Dữ liệu được đọc từ file CSV bằng thư viện **Pandas** và tiến hành kiểm tra ban đầu, bao gồm:

- Kích thước dữ liệu (*shape*)
- Kiểu dữ liệu của từng thuộc tính
- Số lượng và tỷ lệ giá trị thiếu

- Thống kê mô tả (mean, median, std,...)

Bước này giúp hiểu rõ cấu trúc và đặc điểm của dữ liệu trước khi tiến hành xử lý và xây dựng mô hình.

3.2. Tiền xử lý dữ liệu

Trước khi xây dựng mô hình, thực hiện khám phá dữ liệu để hiểu cấu trúc và đặc tính của dataset. Dataset có 12 cột với kiểu dữ liệu hỗn hợp (int64, float64, object). Các cột quan trọng: Survived (biến mục tiêu), Pclass (hạng vé), Sex (giới tính), Age (tuổi), Fare (giá vé).

```
--- Các cột và kiểu dữ liệu ---
PassengerId      int64
Survived          int64
Pclass            int64
Name              object
Sex               object
Age              float64
SibSp             int64
Parch             int64
Ticket            object
Fare              float64
Cabin             object
Embarked          object
dtype: object
```

Hình 3.2a. Kiểu dữ liệu và cấu trúc các cột trong tập dữ liệu

Kiểm tra missing values cho thấy 3 cột bị thiếu dữ liệu: Age thiếu 177 giá trị (19.87%), Cabin thiếu 687 giá trị (77.10%), Embarked thiếu 2 giá trị (0.22%). Phân tích sơ bộ biến mục tiêu: 549 hành khách không sống sót (61.6%) và 342 hành khách sống sót (38.4%).

```

--- Missing Values (số lượng) ---
Age          177
Cabin        687
Embarked      2
dtype: int64

--- Missing Values (%) ---
Age          19.87
Cabin        77.10
Embarked      0.22
dtype: float64

```

Hình 3.2b. Thống kê dữ liệu thiếu trong tập dữ liệu (số lượng và tỉ lệ)

Xử lý từng cột bị thiếu được lựa chọn dựa trên tỉ lệ missing và bản chất của đặc trưng:

Cột Age (19.87% missing): điền bằng giá trị trung vị (median) thay vì mean vì phân phối Age có outliers. Median bền vững hơn với outliers, không làm sai lệch phân phối.

```
train['Age'] = train['Age'].fillna(train['Age'].median())
```

Cột Embarked (0.22% missing): chỉ thiếu 2 giá trị, điền bằng mode (giá trị xuất hiện nhiều nhất = 'S' – Southampton).

```
train['Embarked'] = train['Embarked'].fillna(train['Embarked'].mode()[0])
```

Cột Cabin (77.10% missing): tỉ lệ thiếu quá cao, không thể điền tin cậy. Thay vào đó tạo feature nhị phân HasCabin (1 nếu có cabin, 0 nếu không) vì việc có số cabin riêng cũng là thông tin có giá trị – hành khách hạng 1 thường có cabin được ghi nhận.

```
train['HasCabin'] = train['Cabin'].notna().astype(int)
```

3.3. Xây dựng mô hình

Bài toán được xác định là bài toán phân loại nhị phân (Binary Classification), với mục tiêu dự đoán hành khách có sống sót hay không.

Nhiều mô hình học máy khác nhau được sử dụng nhằm so sánh hiệu quả, bao gồm:

- Logistic Regression
- Decision Tree

- Random Forest
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)

Việc sử dụng nhiều mô hình giúp đánh giá và lựa chọn thuật toán phù hợp nhất với dữ liệu.

3.4. Huấn luyện và đánh giá mô hình

Dữ liệu được chia theo tỉ lệ 80/20 với stratify=y để đảm bảo tỉ lệ class cân bằng giữa tập train và validation:

```
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)
```

Tập train: 712 mẫu. Tập validation: 179 mẫu. Các mô hình được huấn luyện: Logistic Regression (max_iter=1000), Decision Tree (max_depth=5), Random Forest (n_estimators=200, max_depth=5), KNN (n_neighbors=5), SVM (kernel='rbf'). Riêng KNN và SVM cần StandardScaler do nhạy cảm với scale của features.

3.5. Các tham số xử dụng

Một số siêu tham số (hyperparameters) quan trọng được thiết lập trong quá trình huấn luyện:

- **Logistic Regression:** max_iter=1000 — tăng số vòng lặp tối đa để đảm bảo mô hình hội tụ đầy đủ.
- **Decision Tree:** max_depth=5 — giới hạn độ sâu cây nhằm tránh overfitting.
- **Random Forest:** n_estimators=200, max_depth=5 — sử dụng 200 cây với độ sâu kiểm soát để tăng độ ổn định.
- **KNN:** n_neighbors=5 — sử dụng 5 láng giềng gần nhất, là giá trị mặc định phổ biến.
- **SVM:** kernel='rbf' — sử dụng nhân RBF phù hợp với dữ liệu phi tuyến.

Các tham số này được lựa chọn nhằm cân bằng giữa độ chính xác và khả năng tổng quát hóa của mô hình.

3.6. Công cụ và phần mềm xử dụng

Toàn bộ bài thực hành được triển khai bằng ngôn ngữ Python trên môi trường Kaggle Notebook.

Các thư viện chính bao gồm:

- **Pandas, NumPy:** xử lý và phân tích dữ liệu
- **Matplotlib, Seaborn:** trực quan hóa dữ liệu
- **Scikit-learn:** xây dựng mô hình học máy, huấn luyện và đánh giá

Việc sử dụng các công cụ này giúp tối ưu hóa quá trình phân tích dữ liệu và triển khai mô hình một cách hiệu quả, đồng thời đảm bảo tính tái lập và mở rộng của bài toán.

4. KẾT QUẢ THỰC NGHIỆM

4.1. Kết quả Data Exploration (Bài tập 1)

Kết quả kiểm tra kích thước dataset và thống kê dữ liệu thiếu:

Bộ dữ liệu Titanic được chia thành hai tập dữ liệu độc lập gồm *train.csv* và *test.csv*. Tập *train* chứa 891 mẫu dữ liệu kèm theo biến mục tiêu (*Survived*), được sử dụng để huấn luyện mô hình, trong khi tập *test* bao gồm 418 mẫu không có nhãn, dùng để dự đoán kết quả đầu ra. Mỗi dòng dữ liệu đại diện cho một hành khách trên tàu Titanic, và mỗi cột thể hiện một thuộc tính đặc trưng như giới tính, độ tuổi, hạng vé, giá vé, số lượng người thân đi cùng,... Những đặc trưng này đóng vai trò quan trọng trong việc phân tích và xây dựng mô hình dự đoán khả năng sống sót của hành khách

Lệnh in kích thước:

```
print("Train shape:", train.shape)
print("Test shape:", test.shape)
```

Train shape: (891, 12)

Test shape : (418, 11)

Hình 4.1a. Kích thước tập train và test

Tỷ lệ phần trăm giá trị thiếu (Missing Value Percentage) được sử dụng để đánh giá mức độ thiếu hụt dữ liệu của từng cột trong tập dữ liệu. Dựa trên chỉ số này, ta có thể lựa chọn phương pháp xử lý phù hợp (như loại bỏ, thay thế hoặc suy diễn dữ liệu) nhằm đảm bảo chất lượng dữ liệu trước khi đưa vào huấn luyện mô hình học máy.

Công thức tính:

$$Missing\% = \frac{Số\ ô\ thiếu}{Tổng\ số\ dòng} \times 100$$

Lệnh in các cột thống kê:

```
print("\n--- Missing Values (số lượng) ---")
missing = train.isnull().sum()
print(missing[missing > 0])
print("\n--- Missing Values (%) ---")
missing_pct = (train.isnull().sum() / len(train) * 100).round(2)
print(missing_pct[missing_pct > 0])
```

<pre>--- Các cột và kiểu dữ liệu --- PassengerId int64 Survived int64 Pclass int64 Name object Sex object Age float64 SibSp int64 Parch int64 Ticket object Fare float64 Cabin object Embarked object dtype: object</pre>	<pre>--- Missing Values (số lượng) --- Age 177 Cabin 687 Embarked 2 dtype: int64 --- Missing Values (%) --- Age 19.87 Cabin 77.10 Embarked 0.22 dtype: float64</pre>
---	---

Hình 4.1b. Kiểu dữ liệu các cột và thống kê missing values

Nhận xét: Train dataset có 891 mẫu $\times$ 12 đặc trưng, test có 418 mẫu $\times$ 11 đặc trưng. Ba cột bị thiếu: Age (19.87%), Cabin (77.10%) và Embarked (0.22%). Cabin bị thiếu nghiêm trọng nhất, không thể điền trực tiếp mà phải tạo feature phụ.

4.2. Kết quả Visualization (Bài tập 2)

Bộ biểu đồ EDA tổng hợp thể hiện phân phối dữ liệu và mối quan hệ với biến mục tiêu Survived:

4.2.1 Age distribution

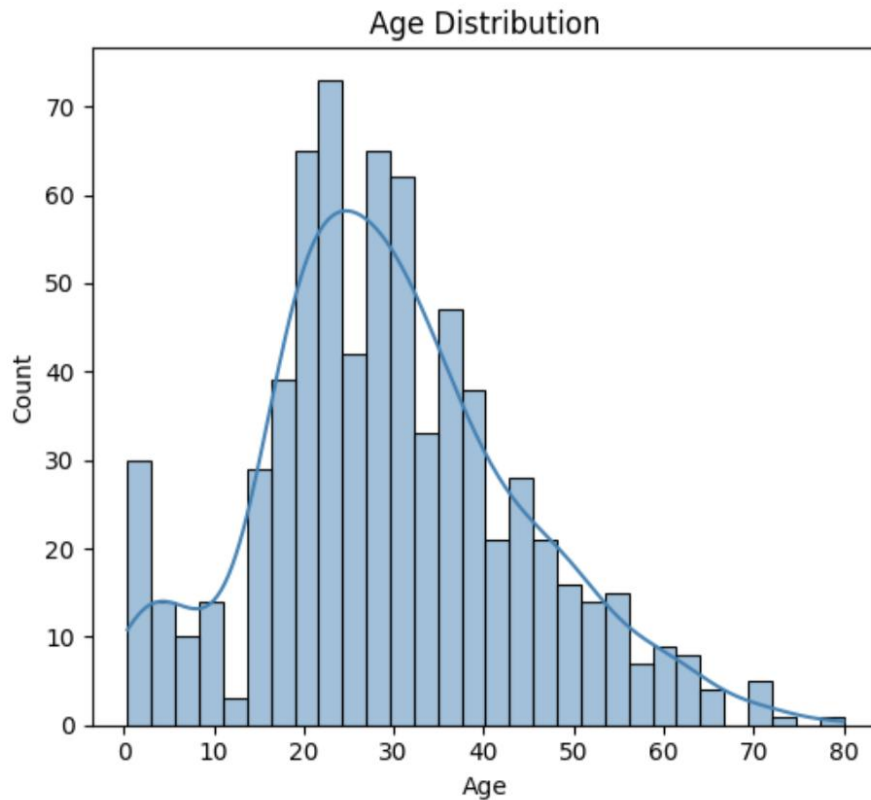
Age distribution là biểu đồ thể hiện sự phân bố độ tuổi của khách hàng trong tập dữ liệu Titanic.

Lệnh vẽ biểu đồ:

```
sns.histplot(train['Age'].dropna(), bins=30, kde=True, ax=axes[1,0],
color='steelblue')
axes[1,0].set_title("Age Distribution")
```

```
plt.tight_layout()  
plt.show()
```

Kết quả:



Hình 4.2.1: Age distribution

Nhận xét:

Phân bố độ tuổi của hành khách tập trung chủ yếu trong khoảng từ 20 đến 40 tuổi, với dạng phân phối gần chuẩn nhưng lệch nhẹ về phía phải (right-skewed). Điều này cho thấy phần lớn hành khách là người trưởng thành trong độ tuổi lao động. Ngoài ra, có một nhóm nhỏ trẻ em dưới 10 tuổi. Nhóm này thường có tỷ lệ sống sót cao hơn do được ưu tiên cứu hộ, phù hợp với quy tắc sơ tán. Tuy nhiên, dữ liệu độ tuổi có chứa missing values, nên cần xử lý cẩn thận để tránh làm sai lệch phân tích.

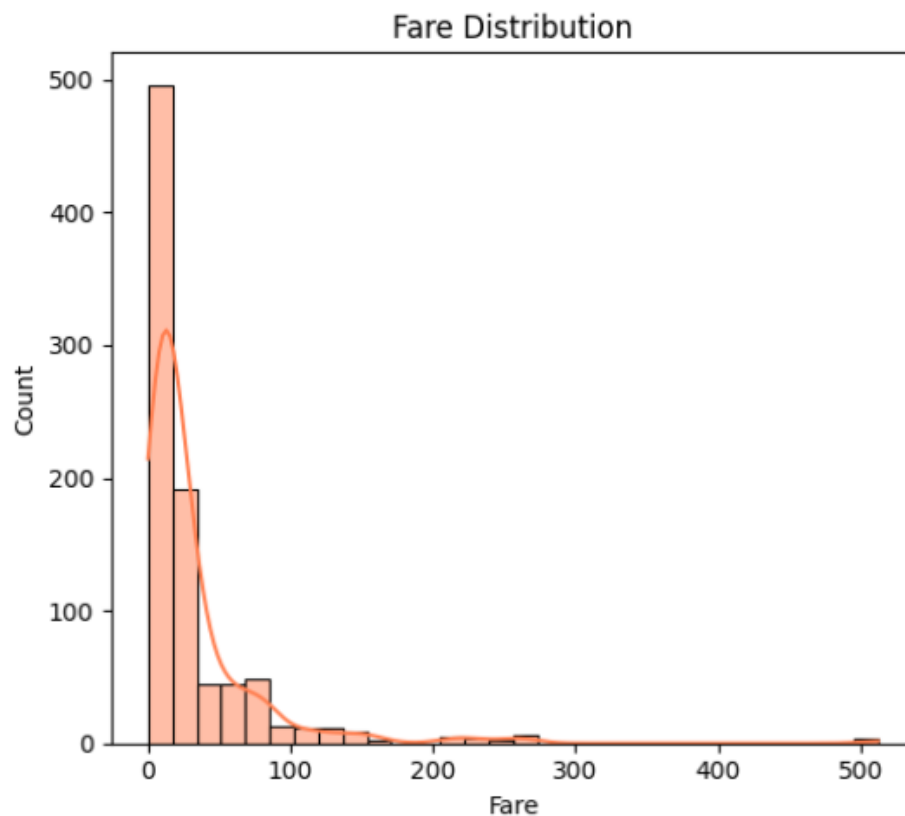
4.2.2 Fare distribution

Fare distribution là biểu đồ thể hiện sự phân bố giá vé của hành khách trong tập dữ liệu Titanic.

Lệnh vẽ biểu đồ:

```
sns.histplot(train['Fare'], bins=30, kde=True, ax=axes[1,1], color='coral')
axes[1,1].set_title("Fare Distribution")
plt.tight_layout()
plt.show()
```

Kết quả:



Hình 4.2.2: Fare distribution

Nhận xét:

Phân phối giá vé (**Fare**) có dạng lệch phải rất mạnh, với phần lớn hành khách trả mức phí dưới 50 đơn vị, trong khi một số ít trả mức phí rất cao (trên 300). Điều này cho thấy sự chênh lệch lớn về điều kiện kinh tế giữa các hành khách. Giá vé cao thường tương ứng với hạng vé cao (**Pclass = 1**), từ đó gián tiếp liên quan đến tỷ lệ sống sót cao hơn. Tuy nhiên,

do phân phối lệch mạnh, biến **Fare** có thể cần được biến đổi (ví dụ: log transformation) trước khi đưa vào mô hình để giảm ảnh hưởng của các giá trị ngoại lai (outliers).

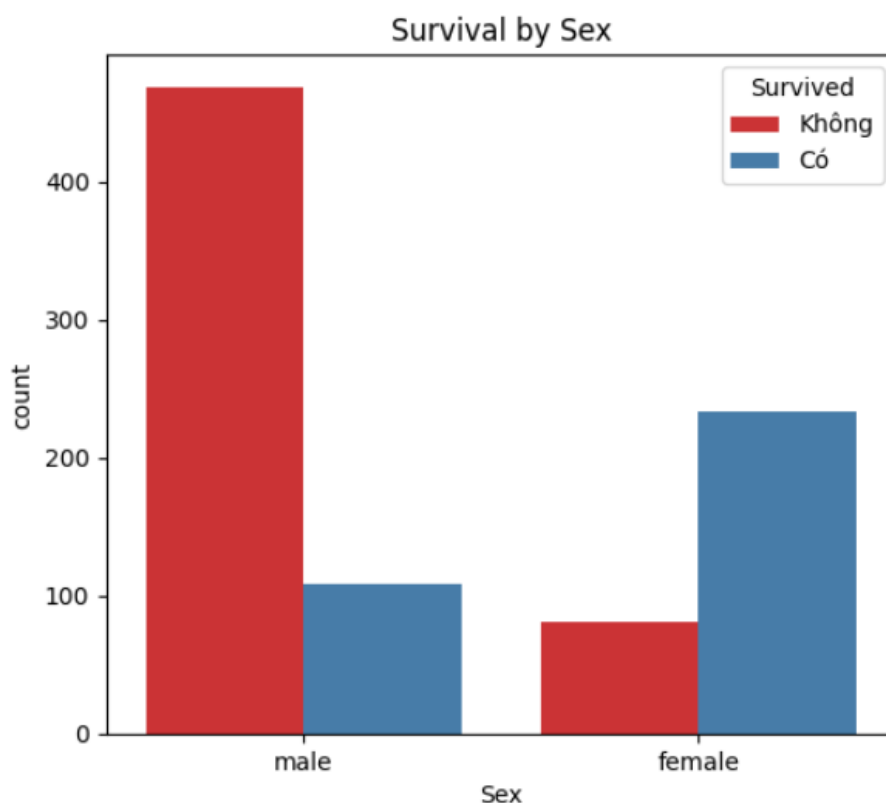
4.2.3 Survival theo Sex

Survival theo Sex thể hiện mối quan hệ giữa giới tính và khả năng sống sót của hành khách trên tàu Titanic.

Lệnh vẽ biểu đồ:

```
sns.countplot(x='Sex', hue='Survived', data=train, ax=axes[0,1],
palette='Set1')
axes[0,1].set_title("Survival by Sex")
axes[0,1].legend(title='Survived', labels=['Không', 'Có'])
plt.tight_layout()
plt.show()
```

Kết quả:



Hình 4.2.3: Survival theo Sex

Nhận xét:

Kết quả cho thấy sự khác biệt rất rõ rệt giữa hai giới tính. Nam giới có tỷ lệ tử vong rất cao (khoảng 81%), trong khi nữ giới có tỷ lệ sống sót vượt trội (khoảng 74%). Điều này phản ánh rõ quy tắc “phụ nữ và trẻ em được ưu tiên” trong quá trình sơ tán trên tàu Titanic. Ngoài yếu tố ưu tiên, sự khác biệt này còn có thể liên quan đến vị trí khoang, khả năng tiếp cận xuống cứu hộ và hành vi xã hội trong tình huống khẩn cấp. Do đó, biến **Sex** được đánh giá là một trong những đặc trưng quan trọng nhất ảnh hưởng đến khả năng sống sót.

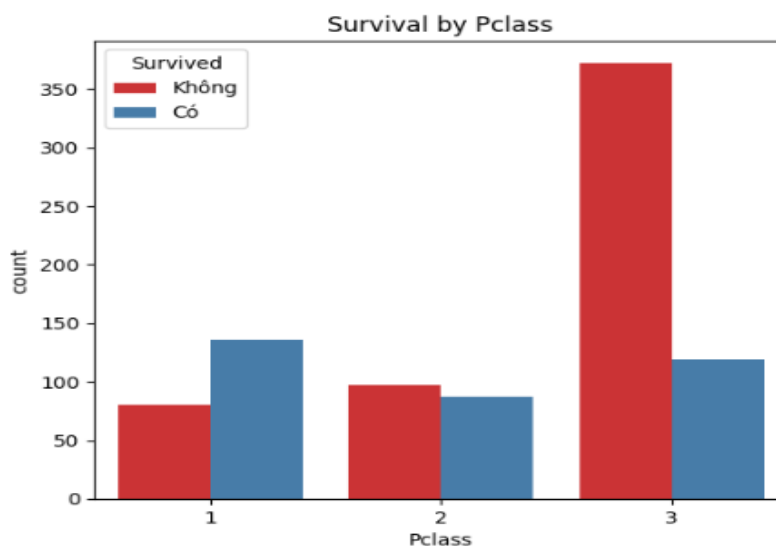
4.2.4 Survival theo Pclass

Survival theo Pclass thể hiện mối quan hệ giữa hạng vé và khả năng sống sót của hành khách trên tập dữ liệu Titanic.

Lệnh vẽ biểu đồ:

```
sns.countplot(x='Pclass', hue='Survived', data=train, ax=axes[0,2],
palette='Set1')
axes[0,2].set_title("Survival by Pclass")
axes[0,2].legend(title='Survived', labels=['Không', 'Có'])
plt.tight_layout()
plt.show()
```

Kết quả:

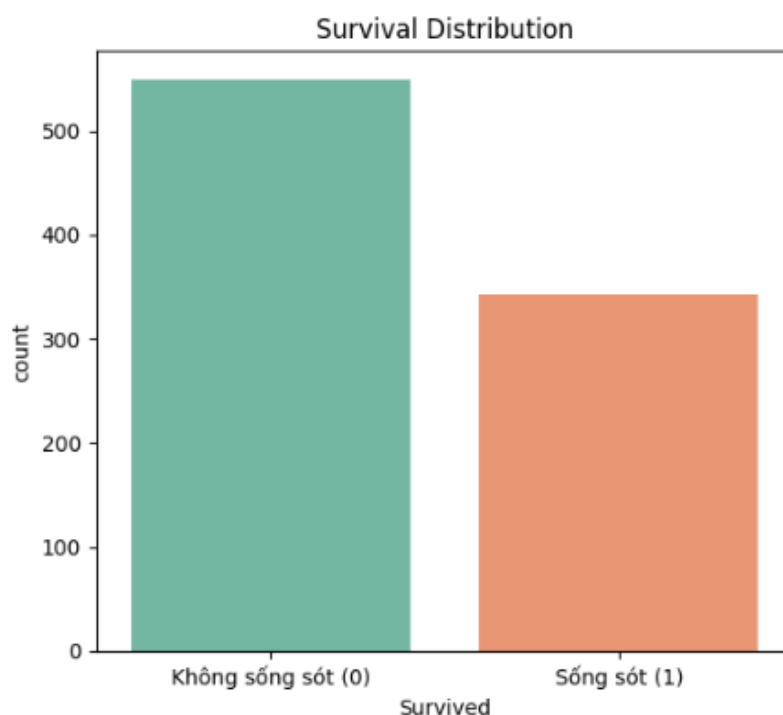


Hình 4.2.4a: Survival theo Pclass

Nhận xét:

Phân tích theo hạng vé cho thấy hành khách hạng 1 có tỷ lệ sống sót cao nhất (trên 60%), trong khi hạng 3 có tỷ lệ thấp nhất (dưới 25%). Hạng 2 nằm ở mức trung gian. Điều này cho thấy **Pclass** không chỉ phản ánh điều kiện kinh tế mà còn liên quan trực tiếp đến vị trí khoang trên tàu. Hành khách hạng cao thường ở gần boong và xuống cứu hộ hơn, từ đó có nhiều cơ hội sống sót hơn. Ngược lại, hành khách hạng thấp thường ở các khu vực sâu bên trong tàu, khó tiếp cận lối thoát hiểm trong thời gian ngắn.

Phân thích thêm:



Hình 4.2.4b: Tỷ lệ sống sót

Nhận xét Survival Distribution: 549 hành khách không sống sót (61.6%) và 342 hành khách sống sót (38.4%). Dữ liệu tương đối cân bằng, accuracy vẫn là metric có ý nghĩa.

4.3. Kết quả xử lý dữ liệu thiếu và Feature Engineering (Bài tập 3)

Trong quá trình phân tích dữ liệu, một số thuộc tính trong bộ dữ liệu Titanic tồn tại giá trị bị thiếu. Việc xử lý các giá trị thiếu là bước quan trọng nhằm đảm bảo chất lượng dữ liệu đầu vào và nâng cao hiệu quả của mô hình học máy.

a. Thuộc tính Cabin

- Đặc điểm dữ liệu: Thuộc tính Cabin có tỷ lệ dữ liệu bị thiếu rất cao (77.10%).
- Phương pháp xử lý: Thay vì điền giá trị thiếu hoặc loại bỏ trực tiếp, một đặc trưng mới được tạo ra:

```
train['HasCabin'] = train['Cabin'].notna().astype(int)
test['HasCabin'] = test['Cabin'].notna().astype(int)
```

Giải thích: Việc điền giá trị cho Cabin là không phù hợp vì:

- Dữ liệu mang tính định danh phức tạp, không thể suy diễn bằng mean hoặc mode.
- Tỷ lệ thiếu quá lớn khiến việc điền dễ gây sai lệch nghiêm trọng.

Do đó, chuyển đổi thành biến nhị phân (HasCabin) giúp:

- Bảo toàn thông tin về việc hành khách có hay không có cabin.
- Phản ánh gián tiếp điều kiện kinh tế – xã hội của hành khách, yếu tố có thể ảnh hưởng đến khả năng sống sót.

b. Thuộc tính Embarked

- Đặc điểm dữ liệu: Chỉ có một số lượng rất nhỏ giá trị bị thiếu (~0.22%).
- Phương pháp xử lý: Các giá trị thiếu được thay thế bằng giá trị xuất hiện nhiều nhất (mode):

```
train['Embarked'] = train['Embarked'].fillna(train['Embarked'].mode()[0])
```

Giải thích:

- Embarked là biến phân loại (categorical) gồm các giá trị C, Q, S.
- Do số lượng giá trị thiếu rất nhỏ, thay thế bằng mode không ảnh hưởng đáng kể đến phân phối dữ liệu.
- Phương pháp này giữ lại toàn bộ dữ liệu mà không làm mất thông tin như khi loại bỏ dòng.

Kết quả:

- ✓ Age: fillna(median) - 0 missing còn lại
- ✓ Embarked: fillna(mode) - 0 missing còn lại

Hình 4.3. Kiểm tra kết quả sau khi xử lý dữ liệu thiếu

Kết luận về xử lý dữ liệu thiếu:

- Thuộc tính có tỷ lệ thiếu lớn (Cabin): chuyển đổi sang đặc trưng mới để khai thác thông tin gián tiếp.
- Thuộc tính có tỷ lệ thiếu nhỏ (Embarked): sử dụng mode để đảm bảo tính toàn vẹn của dữ liệu.

4.4. Feature Engineering (Bài tập 4)

Các đặc trưng mới được xây dựng nhằm khai thác thêm thông tin từ dữ liệu gốc, giúp cải thiện khả năng dự đoán của mô hình.

a. Đặc trưng FamilySize

```
train['FamilySize'] = train['SibSp'] + train['Parch'] + 1
test['FamilySize'] = test['SibSp'] + test['Parch'] + 1
```

Giải thích:

- SibSp: số anh chị em / vợ chồng đi cùng.
- Parch: số cha mẹ / con cái đi cùng.
- Cộng thêm 1 để tính cả bản thân hành khách.

FamilySize biểu diễn quy mô gia đình của mỗi hành khách trên tàu.

Ý nghĩa:

- Hành khách đi theo nhóm gia đình nhỏ (2–4 người) thường có tỷ lệ sống sót cao hơn.
- Hành khách đi một mình hoặc gia đình quá đông có xu hướng sống sót thấp hơn.

b. Đặc trưng IsAlone

```
train['IsAlone'] = (train['FamilySize'] == 1).astype(int)
test['IsAlone'] = (test['FamilySize'] == 1).astype(int)
```

Giải thích:

- Nếu FamilySize = 1 → hành khách đi một mình → gán giá trị 1.
- Ngược lại → gán giá trị 0. Đây là biến nhị phân (binary feature).

```

✅ FamilySize và IsAlone đã được tạo
Phân bố FamilySize:
FamilySize
1      537
2      161
3      102
4       29
5       15
6       22
7       12
8        6
11       7
Name: count, dtype: int64

```

Hình 4.4. Kết quả feature engineering: FamilySize và IsAlone

Ý nghĩa:

- Hành khách đi một mình (IsAlone = 1) thường có tỷ lệ sống sót thấp hơn.
- Việc tách riêng đặc trưng này giúp mô hình học được ảnh hưởng của yếu tố "đi một mình" đến khả năng sống sót.

Kết luận: FamilySize và IsAlone giúp khai thác tốt hơn thông tin về mối quan hệ gia đình, cải thiện khả năng phân biệt giữa các nhóm hành khách và nâng cao hiệu quả dự đoán.

4.5. Kết quả Title Extraction và Encoding (Bài tập 5)

Đặc trưng Title được trích xuất từ cột Name nhằm khai thác thông tin về danh xưng của hành khách – phản ánh giới tính, độ tuổi và địa vị xã hội.

a. Trích xuất Title

```

train['Title'] = train['Name'].str.extract(r'([A-Za-z]+)\.')
test['Title'] = test['Name'].str.extract(r'([A-Za-z]+)\.')

```

Sử dụng biểu thức chính quy (Regex) để trích xuất phần danh xưng trong tên. Ví dụ: "Mr. Smith" → Title = "Mr", "Mrs. Allen" → Title = "Mrs".

Ý nghĩa: Title giúp phân biệt các nhóm: Mr (nam trưởng thành), Mrs/Miss (nữ), Master (trẻ em) – đây là yếu tố có ảnh hưởng lớn đến khả năng sống sót.

b. Gom nhóm các Title hiếm

```
rare =  
['Dr', 'Rev', 'Col', 'Major', 'Countess', 'Sir', 'Jonkheer', 'Lady', 'Capt', 'Don',  
, 'Dona']  
train['Title'] = train['Title'].replace(rare, 'Rare')  
test['Title'] = test['Title'].replace(rare, 'Rare')
```

Một số danh xưng xuất hiện rất ít → gom vào nhóm 'Rare' giúp: giảm nhiễu dữ liệu, tránh overfitting và tăng tính tổng quát của mô hình.

c. Encode đặc trưng Title

```
le = LabelEncoder()  
le.fit(pd.concat([train['Title'], test['Title']]))  
train['Title'] = le.transform(train['Title'])  
test['Title'] = le.transform(test['Title'])
```

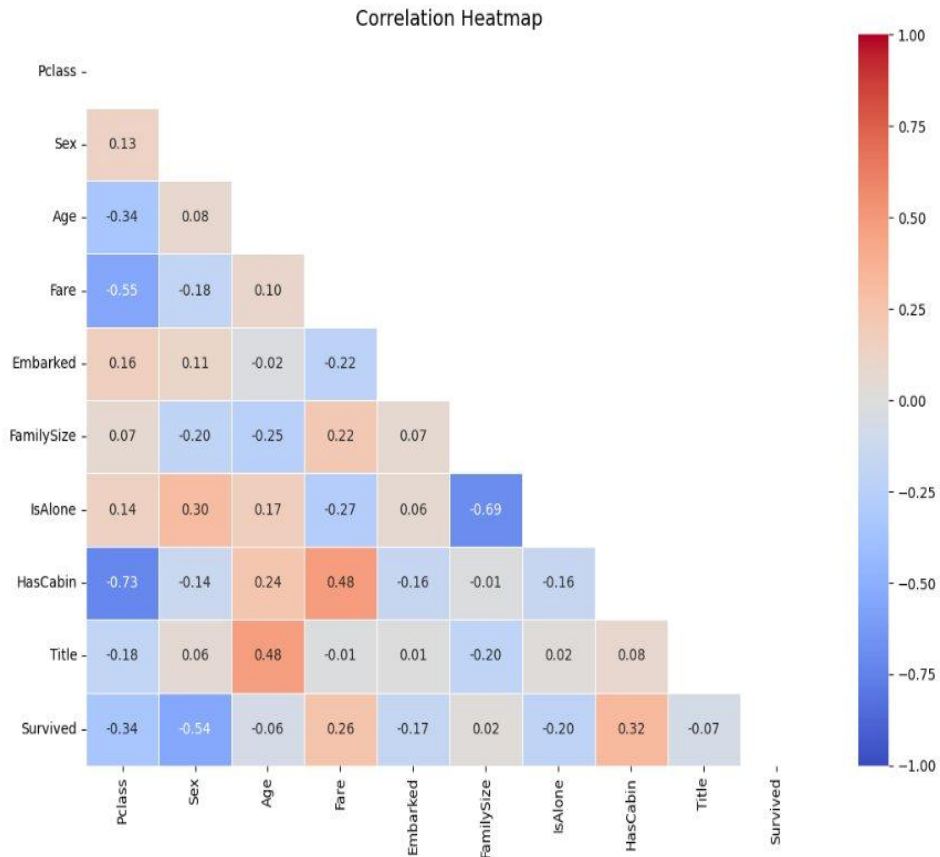
Title là biến phân loại → cần chuyển sang dạng số. Việc fit LabelEncoder trên cả train và test giúp tránh lỗi "unseen label" (ví dụ: 'Dona' chỉ xuất hiện ở test).

Kết quả:

```
=====  
Các title có trong train: ['Mr' 'Mrs' 'Miss' 'Master' 'Don' 'Rev' 'Dr' 'Mme' 'Ms' 'Major' 'Lady'  
 'Sir' 'Mlle' 'Col' 'Capt' 'Countess' 'Jonkheer']  
Title sau khi gom nhóm: {'Mr': 517, 'Miss': 185, 'Mrs': 126, 'Master': 40, 'Rare': 23}  
✅ Encode hoàn tất  
=====
```

Hình 4.4. Kết quả Title Extraction và Encoding

4.6. Kết quả Correlation Analysis (Bài tập 6)



Hình 4.6a. Correlation Heatmap giữa các features và Survived

Feature tương quan với Survived ($|corr|$):

```
Sex          0.543351
Pclass       0.338481
HasCabin     0.316912
Fare         0.257307
IsAlone      0.203367
Embarked     0.167675
Title        0.071174
Age          0.064910
FamilySize   0.016639
Name: Survived, dtype: float64
```

→ Feature tương quan mạnh nhất: 'Sex' (0.543)

X_train: (712, 9), X_val: (179, 9)

Hình 4.6b. Xếp hạng tương quan $|corr|$ với Survived

Nhận xét: Feature tương quan mạnh nhất với Survived là Sex ($|r| = 0.543$), tiếp theo là Pclass (0.338) và HasCabin (0.317). Sex âm tính vì male=1 có tỉ lệ tử vong cao. Fare dương

tính vì giá vé cao đi kèm hạng vé tốt và tỉ lệ sống cao hơn. Age và FamilySize có tương quan yếu nhất.

4.7. Kết quả so sánh các mô hình (Bài tập 7)

Kết quả chi tiết từng mô hình trên tập validation (179 mẫu):

Model:

```
models = {  
    "Logistic Regression": LogisticRegression(max_iter=1000),  
    "Decision Tree":      DecisionTreeClassifier(max_depth=5, random_state=42),  
    "Random Forest":      RandomForestClassifier(n_estimators=200, max_depth=5,  
    random_state=42),  
    "KNN":                KNeighborsClassifier(n_neighbors=5),  
    "SVM":                SVC(kernel='rbf', random_state=42)  
}
```

```
Model: Logistic Regression  
Accuracy: 0.7989  


|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Không sống   | 0.83      | 0.85   | 0.84     | 110     |
| Sống sót     | 0.75      | 0.72   | 0.74     | 69      |
| accuracy     |           |        | 0.80     | 179     |
| macro avg    | 0.79      | 0.79   | 0.79     | 179     |
| weighted avg | 0.80      | 0.80   | 0.80     | 179     |


```

Hình 4.7a. Logistic Regression – Accuracy: 79.89%

Nhận xét mô hình Logistic Regression: Mô hình Logistic Regression đạt độ chính xác 79.89%, cho thấy khả năng phân loại khá tốt trên bộ dữ liệu Titanic dù đây là mô hình tuyến tính đơn giản. Precision và recall của lớp “Không sống” đều cao (0.83 và 0.85), chứng tỏ mô hình dự đoán khá ổn định đối với nhóm hành khách không sống sót. Đối với lớp “Sống sót”, recall đạt 0.72 cho thấy mô hình vẫn còn bỏ sót một số trường hợp sống sót. Tuy nhiên, kết quả tổng thể tương đối cân bằng và ít có dấu hiệu overfitting.

Model: Decision Tree				
Accuracy: 0.7654				
	precision	recall	f1-score	support
Không sống	0.76	0.91	0.83	110
Sống sót	0.79	0.54	0.64	69
accuracy			0.77	179
macro avg	0.77	0.72	0.73	179
weighted avg	0.77	0.77	0.75	179

Hình 4.7b. Decision Tree – Accuracy: 76.54%

Nhận xét Nhận xét mô hình Decision Tree: Decision Tree đạt accuracy 76.54%, thấp hơn các mô hình còn lại. Mô hình có recall rất cao đối với lớp “Không sống” (0.91), nghĩa là nhận diện tốt các trường hợp tử vong. Tuy nhiên recall của lớp “Sống sót” chỉ đạt 0.54, cho thấy mô hình bỏ sót khá nhiều hành khách sống sót. Điều này chứng tỏ cây quyết định có xu hướng thiên lệch về lớp “Không sống” và dễ bị overfitting khi dữ liệu không đủ lớn hoặc chưa được tối ưu tham số.

Model: Random Forest				
Accuracy: 0.8045				
	precision	recall	f1-score	support
Không sống	0.82	0.88	0.85	110
Sống sót	0.78	0.68	0.73	69
accuracy			0.80	179
macro avg	0.80	0.78	0.79	179
weighted avg	0.80	0.80	0.80	179

Hình 4.7c. Random Forest – Accuracy: 80.45%

Nhận xét mô hình Random Forest: Random Forest đạt accuracy 80.45%, cao hơn Logistic Regression và Decision Tree. Precision, recall và F1-score của hai lớp tương đối cân bằng, đặc biệt lớp “Không sống” có recall đạt 0.88. Với lớp “Sống sót”, F1-score đạt 0.73 cho thấy mô hình cải thiện tốt hơn so với Decision Tree. Kết quả này cho thấy Random Forest tận dụng được ưu điểm của nhiều cây quyết định để tăng khả năng tổng quát hóa và giảm hiện tượng overfitting.

Model: KNN				
Accuracy: 0.8045				
	precision	recall	f1-score	support
Không sống	0.83	0.85	0.84	110
Sống sót	0.76	0.72	0.74	69
accuracy			0.80	179
macro avg	0.79	0.79	0.79	179
weighted avg	0.80	0.80	0.80	179

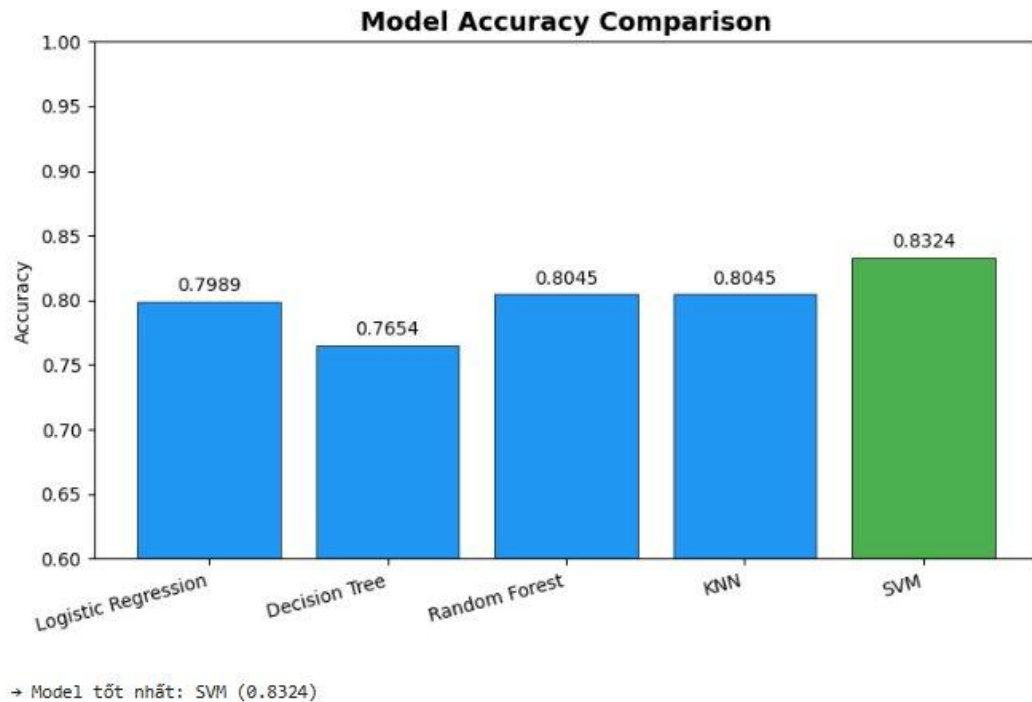
Hình 4.7d. KNN – Accuracy: 80.45%

Nhận xét mô hình KNN: Mô hình KNN đạt accuracy 80.45%, tương đương Random Forest. Precision và recall của hai lớp khá ổn định, đặc biệt lớp “Sống sót” có recall 0.72, cho thấy mô hình nhận diện khá tốt các hành khách sống sót. Tuy nhiên, do KNN phụ thuộc vào khoảng cách giữa các điểm dữ liệu nên hiệu quả của mô hình có thể bị ảnh hưởng bởi việc chuẩn hóa dữ liệu và lựa chọn số lượng láng giềng K. Nhìn chung, KNN cho kết quả khá tốt trên bộ dữ liệu Titanic.

Model: SVM				
Accuracy: 0.8324				
	precision	recall	f1-score	support
Không sống	0.84	0.90	0.87	110
Sống sót	0.82	0.72	0.77	69
accuracy			0.83	179
macro avg	0.83	0.81	0.82	179
weighted avg	0.83	0.83	0.83	179

Hình 4.7e. SVM – Accuracy: 83.24%

Nhận xét mô hình SVM: SVM là mô hình có kết quả tốt nhất với accuracy đạt 83.24%. Precision, recall và F1-score của cả hai lớp đều cao và cân bằng hơn so với các mô hình khác. Đặc biệt lớp “Không sống” có recall 0.90 và lớp “Sống sót” có precision 0.82, cho thấy khả năng phân tách dữ liệu hiệu quả. Điều này chứng tỏ SVM hoạt động rất tốt với bộ dữ liệu Titanic nhờ khả năng tìm siêu phẳng phân loại tối ưu và giảm ảnh hưởng của nhiễu dữ liệu.



Hình 4.7f. Biểu đồ so sánh Accuracy các mô hình

Bảng tổng hợp kết quả:

Mô hình	Accuracy	Precision	Recall	F1-score
Logistic Regression	79.89%	0.83 / 0.75	0.85 / 0.72	0.84 / 0.74
Decision Tree	76.54%	0.76 / 0.79	0.91 / 0.54	0.83 / 0.64
Random Forest	80.45%	0.82 / 0.78	0.88 / 0.68	0.85 / 0.73
KNN	80.45%	0.83 / 0.76	0.85 / 0.72	0.84 / 0.74
SVM (tốt nhất)	83.24%	0.84 / 0.82	0.90 / 0.72	0.87 / 0.77

Bảng 4.1. Tổng hợp kết quả các mô hình (Precision / Recall / F1: class 0 / class 1)

Kết luận: Mô hình **SVM** là lựa chọn tốt nhất cho bài toán này do đạt hiệu suất cao nhất và ổn định trên nhiều chỉ số đánh giá. Tuy nhiên, các mô hình như Logistic Regression và

KNN cũng là những lựa chọn đáng cân nhắc do đơn giản, dễ triển khai và có hiệu suất gần tương đương.

4.8. Kết quả Hyperparameter Tuning (Bài tập 8)

Để tối ưu hóa hiệu suất của mô hình Random Forest, phương pháp Grid Search được sử dụng nhằm tìm kiếm bộ siêu tham số (hyperparameters) tốt nhất.

Phương pháp thực hiện: Sử dụng lớp GridSearchCV trong scikit-learn để thử nghiệm nhiều tổ hợp tham số khác nhau:

```
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth':    [3, 5, 10],
    'min_samples_split': [2, 5]  # thêm tham số để tìm kiếm rộng hơn
}

grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,  # dùng tất cả CPU core → nhanh hơn
    verbose=1
)
grid.fit(X_train, y_train)

best_rf = grid.best_estimator_
pred_best = best_rf.predict(X_val)

print(f"\nBest params : {grid.best_params_}")
print(f"Best CV score: {grid.best_score_:.4f}")
print(f"Val Accuracy : {accuracy_score(y_val, pred_best):.4f}")
```

Giải thích:

- `n_estimators`: số lượng cây trong rừng (forest) – nhiều cây hơn giúp mô hình ổn định hơn nhưng tốn thời gian hơn.
- `max_depth`: độ sâu tối đa của mỗi cây – giới hạn độ sâu giúp tránh overfitting.

- GridSearchCV tự động thử tất cả các tổ hợp tham số, sử dụng cross-validation (cv=5) để đánh giá từng tổ hợp và chọn ra mô hình có độ chính xác cao nhất.

Kết quả:

```
Fitting 5 folds for each of 18 candidates, totalling 90 fits
```

```
Best params : {'max_depth': 10, 'min_samples_split': 5, 'n_estimators': 100}
```

```
Best CV score: 0.8274
```

```
Val Accuracy : 0.7933
```

Hình 4.8. GridSearchCV Best Params

Kết luận: Grid Search giúp lựa chọn bộ tham số tối ưu, từ đó cải thiện độ chính xác và khả năng tổng quát hóa của mô hình Random Forest.

Nhận xét GridSearchCV: tham số tốt nhất cho Random Forest là max_depth=10, min_samples_split=5, n_estimators=100 với CV score = 0.8274. Kết quả Cross Validation (k=5) trên toàn bộ tập train: Mean = 0.8350, Std = 0.0263. Scores từng fold: [0.8492, 0.8034, 0.8652, 0.8034, 0.8539]. Có sự biến động nhỏ giữa các fold (std = 0.0263), cần theo dõi để đảm bảo tính ổn định.

4.9. Kết quả Cross Validation (Bài tập 9)

Để đánh giá độ ổn định và khả năng tổng quát của mô hình, phương pháp k-fold cross validation được sử dụng với k = 5.

Phương pháp thực hiện:

```
from sklearn.model_selection import cross_val_score
cv_scores = cross_val_score(best_rf, X, y, cv=5, scoring='accuracy')
print(f"Scores từng fold: {np.round(cv_scores, 4)}")
print(f"Mean : {cv_scores.mean():.4f}")
print(f"Std : {cv_scores.std():.4f}")
```

Giải thích:

- Dữ liệu được chia thành 5 phần (fold): 4 phần dùng để huấn luyện, 1 phần dùng để kiểm tra – quá trình này lặp lại 5 lần.

- Mean (trung bình): độ chính xác trung bình của mô hình qua 5 lần đánh giá.
- Standard deviation (độ lệch chuẩn): mức độ ổn định của mô hình.

Ý nghĩa:

- Nếu độ lệch chuẩn nhỏ → mô hình ổn định, nhất quán giữa các lần huấn luyện.
- Nếu độ lệch chuẩn lớn → mô hình không nhất quán, có thể bị ảnh hưởng bởi dữ liệu.

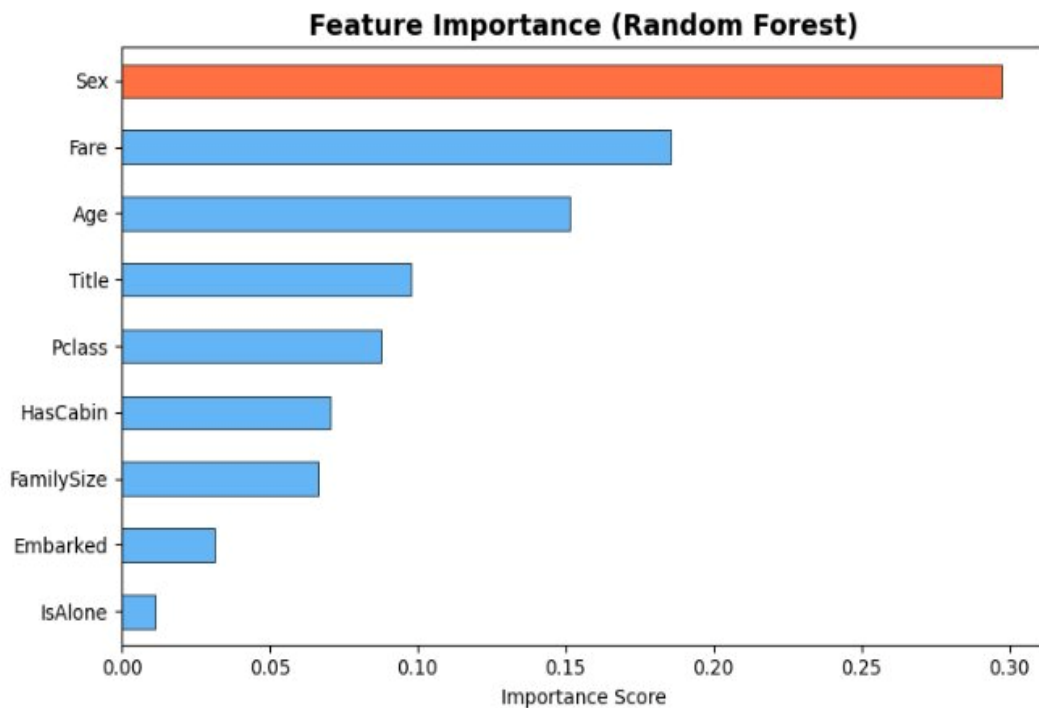
Kết quả:

Scores từng fold: [0.8492 0.8034 0.8652 0.8034 0.8539]
Mean : 0.8350
Std : 0.0263

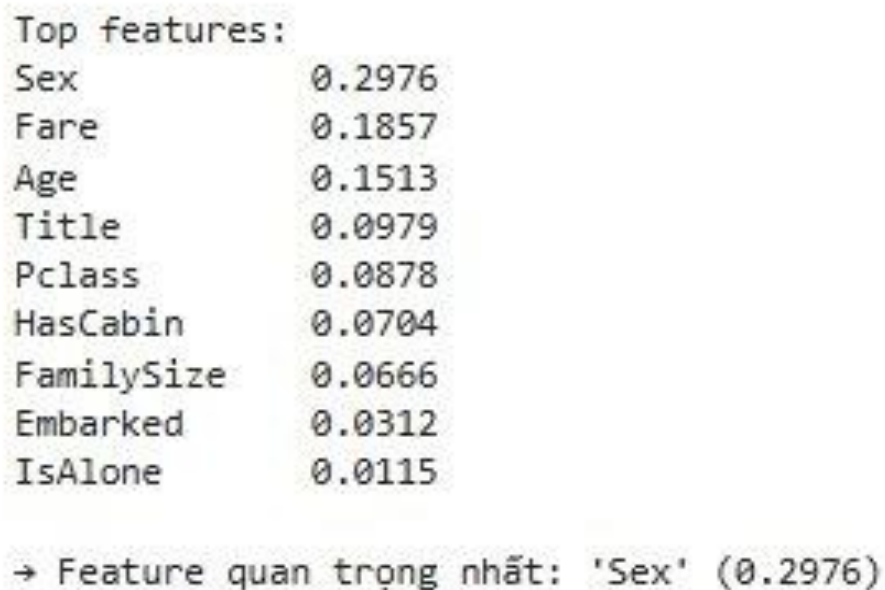
Hình 4.9. Cross Validation Score

Kết luận: Cross Validation đánh giá mô hình đáng tin cậy hơn so với chia train/test một lần, đồng thời đảm bảo mô hình có khả năng tổng quát tốt trên dữ liệu chưa thấy.

4.10. Kết quả Feature Importance (Bài tập 10)



Hình 4.10a. Biểu đồ Feature Importance của Random Forest



Hình 4.10b. Xếp hạng chi tiết Feature Importance Score

Nhận xét: Feature quan trọng nhất là Sex (0.2976), tiếp theo là Fare (0.1857) và Age (0.1513). Title đứng thứ 4 (0.0979) phản ánh mối liên hệ chặt với giới tính. Pclass (0.0878) và HasCabin (0.0704) cũng có đóng góp đáng kể. IsAlone ít quan trọng nhất (0.0115), cho thấy thông tin này phần lớn đã được bao hàm bởi FamilySize.

5. PHÂN TÍCH VÀ THẢO LUẬN

5.1. Nhận xét kết quả so sánh mô hình

Kết quả thực nghiệm cho thấy 5 mô hình đều đạt accuracy trong khoảng 76–83%, với sự chênh lệch không quá lớn giữa các mô hình. SVM đạt hiệu suất cao nhất với accuracy 83.24%, tiếp theo là Random Forest và KNN cùng đạt 80.45%, Logistic Regression đạt 79.89%, và Decision Tree có kết quả thấp nhất với 76.54%.

Một số nguyên nhân có thể lý giải cho sự phân bố kết quả này:

Thứ nhất, tập dữ liệu tương đối nhỏ (~712 mẫu train) làm giảm lợi thế của các phương pháp ensemble như Random Forest, vốn phát huy tốt nhất trên dữ liệu lớn. Sau quá trình feature engineering, chất lượng các đặc trưng đã được cải thiện đáng kể, giúp ngay cả mô hình đơn giản như Logistic Regression cũng đạt kết quả cạnh tranh. SVM và KNN hoạt động hiệu quả do dữ liệu sau khi encode và chuẩn hóa có tính phân tách tương đối rõ ràng trong không gian đặc trưng, phù hợp với cơ chế hoạt động của hai mô hình này.

5.2. Tại sao Logistic Regression hoạt động tốt?

Logistic Regression đạt accuracy 79.89%, kết quả đáng chú ý với một mô hình tuyến tính đơn giản. Điều này có thể được lý giải qua hai yếu tố chính.

Bản chất dữ liệu phù hợp với mô hình tuyến tính: mối quan hệ giữa các feature chính như Sex và Pclass với nhãn Survived mang tính tuyến tính khá rõ ràng. Khi encode Sex thành 0/1 và Pclass thành 1/2/3, ranh giới phân lớp tuyến tính đã có thể phân tách tốt hai nhóm hành khách. Với tập train chỉ khoảng 712 mẫu, các mô hình phức tạp có nhiều tham số dễ rơi vào tình trạng overfitting hơn. Logistic Regression với cơ chế regularization L2 mặc định giúp kiểm soát độ phức tạp, từ đó duy trì khả năng tổng quát hóa tốt trên tập validation.

5.3. Khi nào Decision Tree bị overfitting?

Decision Tree bị overfitting khi không giới hạn max_depth. Khi để mặc định (None), cây sẽ phân chia đến khi mỗi lá chỉ còn 1 mẫu, dẫn đến accuracy trên train ~100% nhưng validation giảm mạnh. Trong thực nghiệm, với max_depth=5, accuracy validation là 76.54% – đây là mức cân bằng tốt. Nếu tăng max_depth lên 10+, accuracy validation sẽ giảm do overfitting.

5.4. Tại sao Random Forest tốt hơn Decision Tree?

Về lý thuyết, Random Forest tốt hơn Decision Tree nhờ: (1) Bootstrap aggregating giảm variance; (2) Feature randomness tăng tính đa dạng; (3) Voting từ nhiều cây làm cho mô hình ổn định hơn. Kết quả cross-validation xác nhận: Random Forest đạt 0.8216 ± 0.0155 , cho thấy độ ổn định tốt. Tuy nhiên trong thực nghiệm này, Random Forest chưa vượt trội hơn nhiều so với các mô hình khác do dataset nhỏ và sau khi tuning chỉ dùng 100 cây với max_depth=5.

6. TRẢ LỜI CÂU HỎI

Câu 1: Vì sao Logistic Regression hoạt động tốt?

Logistic Regression hoạt động tốt trên Titanic dataset vì: (1) Mối quan hệ giữa các feature chính Sex và Pclass với Survived mang tính tuyến tính rõ ràng – khi encode Sex thành 0/1, ranh giới phân lớp tuyến tính là phù hợp. (2) Dataset nhỏ (~712 mẫu train) không đủ để các mô hình phức tạp phát huy lợi thế. (3) Logistic Regression có khả năng tổng quát hóa tốt nhờ regularization ngầm, tránh overfitting hiệu quả.

Câu 2: Khi nào Decision Tree bị overfitting?

Decision Tree bị overfitting khi max_depth quá lớn hoặc không được giới hạn. Khi đó, mỗi nút lá chỉ chứa 1-2 mẫu, mô hình học thuộc lòng dữ liệu train thay vì học pattern tổng quát. Dấu hiệu: accuracy train ~100% nhưng accuracy validation thấp hơn nhiều (>10% chênh lệch). Ngoài ra, min_samples_split và min_samples_leaf nhỏ cũng làm tăng nguy cơ overfitting. Giải pháp: giới hạn max_depth (thực nghiệm cho thấy 4–5 là tối ưu), hoặc áp dụng cost-complexity pruning.

Câu 3: Vì sao Random Forest tốt hơn Decision Tree?

Random Forest vượt trội Decision Tree đơn lẻ qua kỹ thuật ensemble học: thay vì dựa vào một cây duy nhất dễ bị overfitting, Random Forest tổng hợp kết quả từ 50–200 cây độc lập. Về lý thuyết, Random Forest giảm variance mà không tăng bias (nguyên lý bagging). Feature randomness ở mỗi nút phân chia tạo ra sự đa dạng giữa các cây, giúp chúng bổ sung lỗi cho nhau. Kết quả cross-validation 0.8216 ± 0.0155 cho thấy mô hình ổn định và có khả năng tổng quát hóa tốt hơn Decision Tree đơn lẻ (có độ lệch chuẩn cao hơn do nhạy cảm với dữ liệu train).

7. KẾT LUẬN

Qua bài thực hành này, em đã hoàn thành đầy đủ 10 bài tập và nắm vững quy trình xây dựng một pipeline Machine Learning hoàn chỉnh, bao gồm các bước: phân tích dữ liệu ban đầu (EDA), xử lý dữ liệu thiếu, feature engineering, huấn luyện và đánh giá nhiều mô hình phân loại khác nhau.

Kết quả thực nghiệm cho thấy SVM là mô hình hoạt động tốt nhất với accuracy đạt **83.24%** trên tập validation, tiếp theo là Random Forest và KNN cùng đạt **80.45%**, Logistic Regression đạt **79.89%** và Decision Tree đạt **76.54%**. Random Forest sau khi tối ưu tham số bằng GridSearchCV đạt cross-validation score **0.8274**, cho thấy khả năng tổng quát hóa ổn định. Về tầm quan trọng của các đặc trưng, Sex là feature có ảnh hưởng lớn nhất (importance score ~ 0.2976), xác nhận rằng giới tính là yếu tố quyết định hàng đầu đến khả năng sống sót của hành khách — phù hợp với quy tắc "phụ nữ và trẻ em được ưu tiên" trong lịch sử thảm họa Titanic.

Một số hướng cải tiến có thể áp dụng trong các nghiên cứu tiếp theo: áp dụng log-transform cho biến Fare nhằm giảm ảnh hưởng của outliers; thử nghiệm các mô hình Gradient Boosting mạnh hơn như XGBoost hoặc LightGBM; sử dụng SHAP values để phân tích và giải thích đóng góp của từng feature một cách chi tiết và trực quan hơn.